

Funções II: âmbito, composição e testes

Atividade 1 · O mesmo nome em dois âmbitos

Questões de autoavaliação

1. Registrar as saídas e explicar por que o valor exterior permanece inalterado.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original: Dentro: 3; Fora: 10. Alterando a variável local: Dentro: 7; Fora: 10. Com o parâmetro e mostrar(20): Dentro: 20; Fora: 10.

```
x = 10
def mostrar(x):
    print('Dentro:', x)

mostrar(20)
print('Fora:', x)
```

O parâmetro é local à chamada. Nenhuma das atribuições ou chamadas apresentadas reatribui o x global.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 2 · Compor resultados

Questões de autoavaliação

1. Mostrar os passos das duas composições com 4 e explicar o efeito de inverter a ordem.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Com 3, quadrado(dobro(3)) vale 36 e dobro(quadrado(3)) vale 18. Com 4, os resultados são 64 e 32. Cálculos: dobro(4) = 8, quadrado(8) = 64; quadrado(4) = 16, dobro(16) = 32.

```
def dobro(n):
    return n * 2

def quadrado(n):
    return n * n

intermedio = dobro(4)
print(quadrado(intermedio))
intermedio = quadrado(4)
print(dobro(intermedio))
```

As operações não são intercambiáveis: elevar o dobro ao quadrado e dobrar o quadrado são cálculos diferentes.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Funções II: âmbito, composição e testes

Atividade 3 · Isolar cálculos da entrada e saída

Questões de autoavaliação

1. Identificar a parte testável sem input e explicar a separação entre cálculo e leitura.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

O exemplo com preço 100 e taxa 0.20 apresenta Total: 80.0. A chamada direta `preco_final(50, 0.10)` devolve 45.0. Se for impressa antes de `main()`, 45.0 aparece antes das leituras interativas.

`preco_final` pode ser testada com argumentos fixos, sem input. A função `main` coordena a interação. Esta separação permite reutilizar o cálculo e testar casos de forma automática.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 4 · Testar com assert

Questões de autoavaliação

1. Identificar o teste que deteta o erro e explicar por que (0,0) é insuficiente.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

O teste adicional pode ser `assert media(20, 20) == 20`. Ao remover os parênteses, o primeiro teste falha: $10 + 14 / 2$ dá 17.0, não 12. A execução interrompe-se nesse assert, antes da mensagem final.

O caso (0, 0) isolado passaria nas duas versões, porque ambas produzem zero. É necessário usar casos capazes de distinguir o cálculo correto do incorreto.

```
def media(a, b):  
    return (a + b) / 2  
  
assert media(10, 14) == 12  
assert media(0, 0) == 0  
assert media(20, 20) == 20  
print("Testes concluídos")
```

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Funções II: âmbito, composição e testes

Atividade 5 · Aplicação integrada

Questões de autoavaliação

1. Explicar a organização do algoritmo e as decisões tomadas.
2. Comparar resultados previstos e observados em todos os testes pedidos.
3. Identificar uma falha possível e indicar como um teste a poderia revelar.

Resolução e verificação

```
def nota_valida(n):  
    return 0 <= n <= 20  
  
def media(a, b):  
    return (a + b) / 2  
  
def classificar(n):  
    if n >= 10:  
        return 'Aprovado'  
    return 'Reprovado'  
  
def main():  
    a = float(input('Nota 1: '))  
    b = float(input('Nota 2: '))  
    if not (nota_valida(a) and nota_valida(b)):  
        print('Notas inválidas')  
        return  
    m = media(a, b)  
    print('Média:', m)  
    print(classificar(m))  
  
assert nota_valida(0)  
assert nota_valida(20)  
assert not nota_valida(-1)  
assert not nota_valida(21)  
assert media(8, 12) == 10  
assert classificar(10) == 'Aprovado'  
assert classificar(9) == 'Reprovado'  
main()
```

8 e 12: média 10.0, Aprovado. 8 e 10: média 9.0, Reprovado. -1 e 12: Notas inválidas, sem média. 20 e 20: média 20.0, Aprovado. Assume-se entrada numérica. O return antecipado em main termina o processamento de dados inválidos.

Funções II: âmbito, composição e testes

Atividade 6 · Uma dependência global escondida

Questões de autoavaliação

1. Que resultado incorreto surge inicialmente?
2. Como demonstrar que a função corrigida deixou de depender da variável exterior?
3. Por que o parâmetro taxa_desconto era inútil na versão inicial?

Resolução e verificação

```
taxa = 0.10
def calcular(preco, taxa_desconto):
    return preco * (1 - taxa_desconto)

assert calcular(100, 0.20) == 80
taxa = 0.50
assert calcular(100, 0.20) == 80
assert calcular(100, 0) == 100
print('Testes concluídos')
```

Inicialmente obtém-se 90.0; com taxa exterior 0.50 obter-se-ia 50.0. A versão corrigida devolve 80.0 nas duas situações. Os argumentos determinam o resultado. O parâmetro existia na assinatura, mas não era utilizado no cálculo.

Funções II: âmbito, composição e testes

Atividade 7 · Desenhar testes que encontram erros

Questões de autoavaliação

1. Qual o primeiro teste novo que falha?
2. Por que 15 não deteta o defeito?
3. Que par de notas produz exatamente a fronteira de aprovação?

Resolução e verificação

```
def aprovado(media):  
    return media >= 10  
  
def media(a, b):  
    return (a + b) / 2  
  
assert aprovado(9.9) == False  
assert aprovado(10) == True  
assert aprovado(10.1) == True  
assert aprovado(media(8, 12)) == True  
print('Testes concluídos')
```

O teste de 10 falha com `>`. O teste de 15 passa com `>` e `>=`, pelo que não distingue as versões. O par (8,12) produz exatamente 10. Os testes cobrem abaixo, no limite e acima dele, além da composição das duas funções.

Funções II: âmbito, composição e testes

Atividade 8 · Organizar um programa de portes

Questões de autoavaliação

1. Que funções podem ser testadas sem interação?
2. Quais os resultados para os quatro valores?
3. Qual a pré-condição de `calcular_portes` e onde é garantida?

Resolução e verificação

```
def subtotal_valido(valor):  
    return valor >= 0  
  
def calcular_portes(valor):  
    if valor >= 50:  
        return 0  
    return 4  
  
def total_encomenda(valor):  
    return valor + calcular_portes(valor)  
  
def main():  
    valor = float(input('Subtotal: '))  
    if not subtotal_valido(valor):  
        print('Subtotal inválido')  
        return  
    print('Total:', total_encomenda(valor))  
  
assert not subtotal_valido(-1)  
assert subtotal_valido(0)  
assert calcular_portes(49) == 4  
assert calcular_portes(50) == 0  
assert total_encomenda(0) == 4  
assert total_encomenda(50) == 50  
main()
```

Resultados: -1 → Subtotal inválido; 0 → 4.0; 49 → 53.0; 50 → 50.0. As três funções de apoio são testáveis sem input. `calcular_portes` pressupõe subtotal não negativo; `main` verifica isso antes de chamar `total_encomenda`. Os testes diretos devem respeitar o mesmo contrato.

Critério de autoavaliação: 2 pontos quando a implementação cumpre as regras, passa os testes pedidos e as respostas explicam os resultados; 1 ponto se falta parte da solução, dos testes ou da justificação; 0 pontos se a tarefa permanece por resolver. Aplicar o mesmo critério às restantes atividades, num total de 16 pontos. A pontuação orienta o estudo e não constitui classificação formal.